

SmartSpend

AI-Powered Personal Finance Assistant

Joshua Day

ASE 485 — Spring 2026

The Problem

70% of Americans live paycheck-to-paycheck

- People overspend in certain categories without realizing it
- They struggle to create and maintain realistic budgets
- Existing budgeting apps require too much manual input
- Generic financial advice doesn't adapt to individual behavior
- Financial stress is one of the leading causes of anxiety

People need tools that actively help them improve — not just track numbers.

My Solution: SmartSpend

An **AI-powered personal finance assistant** that learns from your actual spending and provides personalized guidance.

Three core ML features:

1. **Auto-categorize** transactions from description text
2. **Generate personalized budgets** from spending history
3. **Produce savings recommendations** by analyzing spending patterns

Plus: budget alerts, persistent auth, dark mode, and a full 11-screen mobile UI

Tech Stack

Layer	Technology
Mobile App	Flutter 3 / Dart — Provider state management
Backend API	Python 3.12 · FastAPI · Uvicorn
Database	PostgreSQL 16 (5 tables, UUID PKs, FK cascades)
ML Engine	scikit-learn (TF-IDF + Multinomial Naive Bayes)
Auth	JWT (PyJWT) + bcrypt (Passlib)
Client Storage	flutter_secure_storage + SharedPreferences
Containers	Docker Compose (API + DB)
Testing	mocktail (Flutter) + pytest (backend)

Why This Stack?

- **Flutter** — single codebase for iOS, Android, and web; fast iteration with hot reload
- **FastAPI** — async Python framework with automatic OpenAPI docs; ideal for ML integration since scikit-learn is Python-native
- **PostgreSQL** — relational integrity with foreign keys and cascading deletes; UUID PKs for distributed safety
- **Docker Compose** — one command spins up the entire backend; reproducible across machines
- **Provider** — lightweight state management that scales well for 6 providers without boilerplate

Architecture

```
Flutter App (11 screens)
├── 6 Providers (ChangeNotifier + Provider)
│   └── 6 Services (HTTP)
│       ├── ApiClient (JWT injection, timeout, error handling)
│       │   └── FastAPI Backend (:8000)
│       │       ├── Auth Router (register, login – JWT + bcrypt)
│       │       ├── CRUD Routers (transactions, budgets, goals, recs)
│       │       ├── ML Router (/ml/categorize, /ml/budgets/generate,
│       │       │           /ml/recommendations/generate)
│       │       ├── ML Engine (TF-IDF + Naive Bayes pipeline)
│       │       └── PostgreSQL 16 (psycopg2 connection pool)
```

Data Model

Five core tables — all UUID primary keys, `ON DELETE CASCADE` foreign keys:

Table	Purpose
users	Accounts with bcrypt-hashed passwords
transactions	Spending records with amount, category, description, date
budgets	Per-category budget limits with configurable periods
goals	Savings goals with target amounts and progress tracking
recommendations	AI-generated savings tips with potential savings amounts

Schema: `docker/init.sql` · Seed data: `docker/seed.sql`

API Endpoints — 18 Total

Group	Endpoints	Auth
Health	GET /health	—
Auth	POST /auth/register , POST /auth/login	—
Transactions	GET , POST , PUT , DELETE	JWT
Budgets	GET , POST , PUT , DELETE	JWT
Goals	GET , POST , PUT , DELETE	JWT
Recommendations	GET	JWT
ML	POST /ml/categorize , POST /ml/budgets/generate , POST /ml/recommendations/generate	JWT

ML Feature 1: Transaction Categorization

- **Pipeline:** TF-IDF vectorizer → Multinomial Naive Bayes classifier
- **Training:** 150+ labeled descriptions across **8 categories** (Food, Transport, Shopping, Entertainment, Bills, Healthcare, Income, Other)
- **How it works:** user adds a transaction without selecting a category → the ML engine predicts the category from the description text with a confidence score
- **Accuracy:** meets the 80% target on representative test data
- **Endpoint:** `POST /ml/categorize` (standalone) + auto-integrated into `POST /transactions`

ML Feature 2: Budget Generation

- Analyzes the user's full **transaction history per category**
- Computes average monthly spend and adds a **10% buffer** for realistic budgets
- `POST /ml/budgets/generate` replaces existing budgets with ML-suggested ones
- One-tap "**Generate Budgets**" button in the Flutter Budget screen
- Solves the problem of users not knowing where to start with budgeting

ML Feature 3: Savings Recommendations

Rule-based ML engine evaluating **5 spending patterns**:

1. **Over-budget categories** — flags categories exceeding their budget limit
2. **Spending spikes** — detects month-over-month increases > 20%
3. **Missing budgets** — identifies categories with spending but no budget set
4. **High income ratio** — warns when total spending exceeds 80% of income
5. **Top expense categories** — suggests cuts in the largest spending areas

`POST /ml/recommendations/generate` with one-tap refresh in the app

Budget Alerts

- **Client-side alert computation** based on current spending vs. budget limits
- **Warning** at 80% usage (yellow) · **Danger** at 100% (red)
- Alert banners displayed on the **Home dashboard**
- **Notification bell** with badge count showing active alerts
- **Bottom-sheet detail view** listing all alerts with amounts and categories
- Alert cards on the **Budget overview screen**

Flutter Frontend — 11 Screens

Screen	Purpose
Login / Register	JWT auth with form validation
Home Dashboard	Summary cards, budget alert banners, notification bell
Transactions	List with category filter + add transaction form
Budget	Per-category budgets with ML generate button
Goals	Savings goals with progress bars
Analytics	Category breakdowns, period selector, month-over-month
Recommendations	AI-powered savings tips with refresh
Settings	Dark mode, notifications, biometrics, currency
Account	Profile and navigation hub

State Management — 6 Providers

Provider	Responsibility
SettingsProvider	Dark mode, notifications, currency locale (SharedPreferences)
AuthProvider	Login, register, logout, session restore (secure storage)
TransactionProvider	CRUD + category filtering for transactions
BudgetProvider	CRUD + ML budget generation
GoalProvider	CRUD + progress tracking for savings goals
RecommendationProvider	Fetch + ML-refresh recommendations

All registered in `MultiProvider` with `ApiClient` singleton injected via `Provider`

Testing

Flutter (25+ test files)

- **Model tests** — `fromJson` / `toJson` round-trips and computed properties
- **Provider tests** — loading state, success, and error paths with mocktail mocks
- **Service tests** — all 6 services tested against mocked HTTP responses
- **Widget tests** — SummaryCard, TransactionTile, GoalProgressCard, CategoryCard, LoadingOverlay, BudgetAlertBanner
- **Utility tests** — validators, error helpers
- **Integration test** — full app launch, login screen, form validation

Backend (pytest)

- All CRUD and ML endpoints tested with mocked database layer
- ML engine unit tests — categorization accuracy, budget generation.

Full Project Timeline

Week	Sprint 1 Milestone
4	Project setup — GitHub, Docker, PostgreSQL, Flutter & FastAPI scaffolding
5	User auth — JWT, bcrypt, login/register screens, DB schema
6	Transactions — CRUD API + Flutter list & add screens
7	Dashboard — budgets/goals/recs API + analytics screens
8	Testing, bug fixes, UI polish, Sprint 1 Presentation

Full Project Timeline (cont.)

Week	Sprint 2 Milestone
9	Persistent auth, SettingsProvider, expanded test suite (25+ files)
10	Live API wiring — all 11 screens connected to real providers
11	ML categorization engine (TF-IDF + Naive Bayes)
12	ML budget generation + ML savings recommendations
13	Budget alerts (banners, notification bell, bottom sheet)
14	Backend pytest suite, integration tests, UI polish
15	Final testing, deployment prep, Final Presentation

Acceptance Criteria — All Met

Criteria	Status
Users can input spending data and see it categorized	Done — ML auto-categorization
ML model generates budgets reflecting spending patterns	Done — <code>/ml/budgets/generate</code>
Savings recommendations relevant to top spending categories	Done — rule-based ML engine
Alerts trigger when approaching budget limits	Done — 80% warning, 100% danger
ML categorization at least 80% accuracy	Met — NB + TF-IDF pipeline
API responds within 2 seconds	Met — typical response <

Final Metrics

- **Total Lines of Code:** ~6,500+
- **Backend Endpoints:** 18 (15 CRUD + 3 ML)
- **Flutter Screens:** 11
- **Providers:** 6
- **Flutter Test Files:** 25+
- **Backend Test Files:** pytest suite covering all endpoints + ML engine
- **Total Features Delivered:** 19 / 19
- **Total Requirements Met:** 20 / 20
- **Sprints Completed:** 2 / 2

Demo Flow

1. **Login** as demo user → Home dashboard with budget alert banners
2. **Add Transaction** without selecting a category → ML auto-categorizes
3. **Budget Screen** → tap "Generate" → ML creates budgets from history
4. **Recommendations** → tap refresh → ML analyzes spending and generates tips
5. **Notification Bell** → badge shows alert count, bottom sheet lists details
6. **Analytics** → category breakdowns with period selector
7. **Dark Mode** → toggle in Settings, persisted across restarts

What Went Well

- Full-stack architecture from Sprint 1 made Sprint 2 ML integration smooth
- Provider pattern kept state management clean across 11 screens
- Docker Compose made the backend reproducible — one command to spin up everything
- ML pipeline (TF-IDF + NB) is fast and lightweight — no GPU required
- Budget alerts add real user value with minimal additional code
- Test coverage stayed strong throughout both sprints

What Was Challenging

- Connecting Flutter to FastAPI through Docker networking took trial and error
- JWT token refresh and persistent session restoration had edge cases
- Designing the ML training corpus to cover enough transaction description variety
- Balancing rule-based vs. pure ML approaches for recommendations
- Keeping the Docker image lean after adding scikit-learn + numpy

What I Learned

- **Full-stack development** end-to-end: mobile client, REST API, database, ML, and containerization
- **ML integration** in a production-style app — not just a notebook, but a real API endpoint
- **Provider architecture** at scale — 6 providers, 6 services, clean separation of concerns
- **Docker Compose** for local development — health checks, volumes, networking
- **Professional workflow** — sprint planning, weekly updates, presentations, testing

Future Enhancements

- CSV / bank feed import for transaction data
- Firebase push notifications for real-time budget alerts
- Adaptive budgets that adjust month-to-month automatically
- Deep learning categorization (BERT embeddings) for higher accuracy
- Multi-user household support with shared budgets

Thank You

Questions?

Joshua Day — dayj16@mymail.nku.edu

Repository: github.com/Jolteer/ase485-capstone-finance-ml